// Assurance Assessment

Sep 2, 2026 - Sep 5, 2026

Earn v2 1.7 MultiDepositAsyncVault

Blueprint Finance

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

DATE OF ENGAGEMENT
Sep 2, 2026 - Sep 5, 2026

SEVERITY BREAKDOWN

8	5	1	2
All Findings	Solved	Risk Accepted	Acknowledged



INTRODUCTION

`Blueprint Finance` engaged `Halborn` to perform a security assessment of their smart contracts from September 2nd, 2026 to September 4th, 2026. The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

The `Blueprint Finance` codebase in scope consists of changes made to the Earn V2 vault protocol that add a multi-asset deposit async vault accepting whitelisted non-base assets while keeping base-asset share accounting, a PRE_MULTI_DEPOSIT lifecycle hook with a per-token lifetime deposit-cap hook, an OracleModule Chainlink feed registry with one-to-one and oracle-based asset pricers, and EIP-170 codesize reductions that extract vault hot paths into libraries.

ASSESSMENT SUMMARY

`Halborn` was allocated 3 days for this engagement and assigned 1 full-time security engineer to conduct a comprehensive review of the smart contracts within scope. The engineer is an expert in blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment are to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, `Halborn` identified several areas for improvement to reduce the likelihood and impact of security risks, which were partially addressed by the `Blueprint Finance` team. The main recommendations were:

- `Price assets that can deviate from the base asset through OracleAssetPricer with tight OracleModule price caps and staleness bounds, reserve OneToOneAssetPricer for hard-pegged assets, and add a live peg-band check to OneToOneAssetPricer.convertToBase() that reverts when the ratio leaves parity.`
- `Restrict the PRE_DEPOSIT and POST_DEPOSIT hook invocations in MultiAssetVaultHelperLib.multiAssetDeposit() to genuine base-asset deposits so non-base deposits drive only the PRE_MULTI_DEPOSIT hook.`
- `Add the validateCaps and validateStaleness flags to the OracleSet event and emit them in OracleModule.setOracle() so off-chain monitors can tell an enforced oracle configuration from one whose price-bound and staleness checks are disabled.`

TEST APPROACH AND METHODOLODY

`Halborn` conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.
The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose and use.
- Manual code review and walkthrough of smart contracts to identify any logical issues.
- Comprehensive assessment of the safety and usage of critical Solidity variables and functions within scope that could lead to arithmetic-related vulnerabilities.
- Local testing using custom scripts (`Foundry`).
- Fork testing against main networks (`Foundry`).
- Static security analysis of scoped contracts, and imported functions (`Slither`).

Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational
9 - 10	7 - 8.9	4.5 - 6.9	2 - 4.4	0 - 1.9

Scope

URL

(a) URL:
https://github.com/Blueprint-Finance/earn-v2-core/compare/9531bfd60c36aca84fd2f25be8e5e25a6fd9c05e...15db3b58f5341608fcb39dc20c03e25b8f158b73

(b) Items in scope:

- src 22 files
  - implementation 2 files
    - ConcreteMultiAssetDepositAsyncVault.sol Solidity
    - ConcreteStandardVaultImpl.sol Solidity
  - interface 4 files
    - IAssetPricer.sol Solidity
    - IConcreteMultiAssetDepositAsyncVault.sol Solidity
    - IHook.sol Solidity
    - IMultiAssetDepositStrategy.sol Solidity
  - lib 6 files
    - storage 1 file
      - ConcreteMultiAssetDepositAsyncVaultStorageLib.sol Solidity
    - Hooks.sol Solidity
    - MultiAssetVaultHelperLib.sol Solidity
    - StandardVaultHelperLib.sol Solidity
    - StateSetterLib.sol Solidity
    - UserActionsLib.sol Solidity
  - periphery 8 files
    - hooks 3 files
      - BaseHook.sol Solidity
      - HookContainer.sol Solidity
      - MultiAssetDepositCapHook.sol Solidity
    - lib 2 files
      - TimelockConfigurationLib.sol Solidity
      - TimelockContractTypesLib.sol Solidity
    - oracle 1 file
      - OracleModule.sol Solidity
    - strategies 1 file
      - MultiAssetStrategy.sol Solidity
    - timelock 1 file
      - Timelock.sol Solidity
  - pricer 2 files
    - OneToOneAssetPricer.sol Solidity
    - OracleAssetPricer.sol Solidity

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID:

- fa58320ceb92916f5d4b5dbb25fbcac78acc9cf8
- 8844433b123d2c12e0d47e7b5ecead5cfa5b2bba
- 5db8b9da2cdabc996fc28617e4dffc134feb5a8f
- 61b64e9fd421e05ee2d68a1949386af6720747b2
- 1454aab1cb6bc9043d11900de6fd7ecb4e41155f

Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	Excess share minting from par valuation of a depegged deposit asset	Low	2.5	RISK ACCEPTED 09/18/2026
HAL-002	Non-base deposits invoke the base-asset PRE_DEPOSIT and POST_DEPOSIT hooks in addition to PRE_MULTI_DEPOSIT	Informational	0.8	SOLVED 09/18/2026
HAL-003	Obscured price check enforcement due to incomplete OracleSet event	Informational	0.5	SOLVED 09/17/2026
HAL-004	Missing standard Deposit event on the non-base deposit path	Informational	0.5	SOLVED 09/17/2026
HAL-005	Silent governance tier reset due to unconditional reinitializer reseeding	Informational	0.4	ACKNOWLEDGED 09/18/2026
HAL-006	Owner gated controls temporarily unreachable due to unhardened ownership transfer	Informational	0.4	SOLVED 09/17/2026
HAL-007	Non-base deposit path blocked due to unenforced hook container alignment	Informational	0.4	ACKNOWLEDGED 09/18/2026
HAL-008	Non-base deposits blocked due to unsynchronized multi asset strategy pointer	Informational	0.4	SOLVED 09/17/2026

Findings & Tech Details

HAL-001 RISK ACCEPTED

Excess share minting from par valuation of a depegged deposit asset

Low · 2.5 AO:A/AC:L/AX:H/R:N/S:U/C:N/A:N/I:N/D:H/Y:N

DESCRIPTION

The `OneToOneAssetPricer.convertToBase()` function is the pricer that `ConcreteMultiAssetDepositAsyncVault` uses to value non-base deposits for assets that are intended to be economically 1:1 with the vault's base asset, such as same peg stablecoins. Every non-base deposit flows through `MultiAssetVaultHelperLib.multiAssetDeposit()`, which calls `prepareNonBaseDeposit()` and in turn the configured pricer's `convertToBase()` to derive the base value that determines how many shares are minted to the depositor.

However, `OneToOneAssetPricer.convertToBase()` returns the deposit amount adjusted only for the difference in token decimals and never reads any price feed. It reports a strict 1:1 value even when the whitelisted deposit asset has lost its peg and trades below the base asset on the market. Because the vault mints shares directly from that base value and the deposited token is forwarded to the strategy at the same par valuation, the pricer overcredits any deposit made while the asset is depegged.

A depositor who acquires a depegged whitelisted asset cheaply on the open market can deposit it at par and mint shares worth the full base value. If a base asset of 1,000,000 backs the pool and an attacker deposits 1,000,000 units of a whitelisted asset trading at 0.90, the vault credits 1,000,000 base units and mints shares as if 1,000,000 base had been supplied. The attacker then redeems those shares for real base liquidity from the shared pool, extracting the peg gap and leaving the remaining shareholders holding the devalued asset. The extractable amount is bounded by the depeg magnitude multiplied by the deposit size and requires the whitelisted asset to be depegged at deposit time.

RECOMMENDATION

Value assets that can deviate from the base asset against a live price source by configuring `OracleAssetPricer` for them, and set the `OracleModule` min and max price caps and staleness parameters tightly so a depegged or stale price is rejected rather than credited at par. Reserve `OneToOneAssetPricer` for assets that remain hard pegged, and treat the existing instant `ConcreteMultiAssetDepositAsyncVault.removeWhitelistedAsset()` as the incident response to delist an asset the moment it depegs.

Additionally, add a live peg band check to `OneToOneAssetPricer.convertToBase()` that reads both legs from `OracleModule` and reverts when the ratio deviates from parity beyond a tight configured bound.

REMEDIATION COMMENT

RISK ACCEPTED: The `Blueprint Finance` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> We acknowledge the issue. `OneToOneAssetPricer.convertToBase()` values a non-base deposit at par (decimals only) and will over-credit shares if a whitelisted asset is depegged at deposit time. That risk is accepted for this pricer.
> OneToOneAssetPricer is reserved for same-peg stablecoins. Pegs are monitored off-chain. If an asset breaks, we pause the vault and delist it with `removeWhitelistedAsset()`. Both `pause()` and delist are immediate and not timelocked; deposits and the async withdrawal/claim path stop while paused.
> We will not add an `OracleModule` peg band to `OneToOneAssetPricer`. That would duplicate `OracleAssetPricer`, and USD stablecoin feeds often stay near par through a depeg, so the check would not reliably close this path. Pause, delist, and pricer choice remain the control.

HAL-002 SOLVED

Non-base deposits invoke the base-asset PRE_DEPOSIT and POST_DEPOSIT hooks in addition to PRE_MULTI_DEPOSIT

Informational · 0.8 AO:A/AC:L/AX:H/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `MultiAssetVaultHelperLib.multiAssetDeposit()` function is the non-base deposit path for `ConcreteMultiAssetDepositAsyncVault`. The design separates base-asset and non-base-asset deposit accounting onto distinct rails: base-asset deposits run the inherited ERC-4626 path and the `PRE_DEPOSIT` lifecycle hook, while non-base deposits are meant to be tracked and capped through the dedicated `PRE_MULTI_DEPOSIT` hook and `MultiAssetDepositCapHook`.

However, `multiAssetDeposit()` calls `UserActionsLib.preDeposit()` (which fires the `PRE_DEPOSIT` hook with `assets` set to the base-equivalent value) and `UserActionsLib.postDeposit()` (which fires `POST_DEPOSIT`), in addition to the multi-asset-only `PRE_MULTI_DEPOSIT` hook. A non-base deposit therefore drives every base-asset lifecycle hook as if a real base-asset deposit had entered the vault's idle balance, even though no base asset was received.

Any stateful hook wired to `PRE_DEPOSIT` that assumes it only observes real base-asset inflows, such as a cumulative base-deposit cap or a lock/allowlist gate, is also consumed and enforced by non-base deposits valued at their base-equivalent amount. This can silently apply a base-asset cap to multi-asset flows or block multi-asset deposits, contradicting the documented separation of the two deposit rails. No fund loss was identified; the impact depends on which hooks the operator installs.

RECOMMENDATION

Align the hook firing with the intended two-rail model. Confirm whether base-asset lifecycle hooks are meant to observe non-base deposits, and if not, restrict the `PRE_DEPOSIT` and `POST_DEPOSIT` invocations to genuine base-asset deposits so that non-base deposits drive only the `PRE_MULTI_DEPOSIT` hook.

If the dual firing is intended, document that `PRE_DEPOSIT` and `POST_DEPOSIT` hooks must treat the `assets` value on a multi-asset deposit as a base-equivalent notional rather than received base liquidity.

REMEDIATION COMMENT

Addressed in: fa58320ceb92916f5d4b5dbb25fbcac78acc9cf8

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by documenting that the `PRE_DEPOSIT` and `POST_DEPOSIT` hooks deliberately fire on the non-base

deposit path with `assets` carried as a base-asset-equivalent notional, across the `IHook` natspec, the `multiAssetDeposit()` firing order, the vault natspec, and `Architecture.md`.

HAL-003 SOLVED

Obscured price check enforcement due to incomplete OracleSet event

Informational · 0.5 `AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N`

DESCRIPTION

The `OracleModule.setOracle()` function emits the `OracleSet` event with the asset, feed, `minPrice`, `maxPrice`, and `maxDelay`, so off-chain monitors can observe how an oracle was configured.

However, the event omits `validateCaps` and `validateStaleness`, the two booleans that decide whether the price-bound and staleness checks are enforced at all in `_validateOracleFeed`. Nothing forces the numeric fields to zero when a boolean is false, so an operator can pass plausible-looking bounds alongside `validateCaps_ = false` and `validateStaleness_ = false`, and the emitted event is indistinguishable from a fully enforced configuration.

An event-driven monitor, indexer, or auditor watching `OracleSet` cannot tell an enforced config from one whose safety checks are disabled without separately reading the public `oracleConfigs(asset)` getter for every event. This does not change value flow, but it weakens off-chain detection of a security-relevant configuration choice.

RECOMMENDATION

Add `validateCaps` and `validateStaleness` to the `OracleSet` event signature and emit them in `OracleModule.setOracle()`.

REMEDIATION COMMENT

Addressed in: 8844433b123d2c12e0d47e7b5ecead5cfa5b2bba

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by adding the `validateCaps` and `validateStaleness` booleans to the `OracleSet` event and emitting them in `setOracle()`, so an event-driven monitor can distinguish an enforced feed configuration from one with the checks disabled without reading `oracleConfigs` per event.

HAL-004 SOLVED

Missing standard Deposit event on the non-base deposit path

Informational · 0.5 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

The `MultiAssetVaultHelperLib.multiAssetDeposit()` function is the non-base deposit entrypoint of the multi-asset vault. It mints real ERC-4626 shares to the receiver through `ERC20_OZ_5_2_0_Lib._mint`, whose update emits only the ERC-20 `Transfer` event, and separately emits the custom `MultiAssetDeposit` event.

However, it never emits the standard `IERC4626.Deposit` event, whereas the base asset path `UserActionsLib.executeDeposit()` always does. Because the vault inherits and advertises the full `IERC4626` interface, off-chain consumers reasonably treat `Deposit` as the canonical share issuance signal for every deposit.

As a result, any indexer, subgraph, dashboard, or reconciliation that derives share issuance from summed `Deposit` events silently under-counts every non-base deposit, and the vault does not fully conform to the ERC-4626 event surface it exposes. Internal accounting is unaffected because it relies on cached totals rather than events, so the impact is confined to off-chain observability and standards conformance.

RECOMMENDATION

In `MultiAssetVaultHelperLib.multiAssetDeposit()`, emit `IERC4626.Deposit(sender, receiver, baseAllocated, shares)` using the base-equivalent value already computed, in addition to the custom `MultiAssetDeposit` event, and document in NatSpec that `MultiAssetDeposit` carries the raw deposit-asset detail while `Deposit` is the canonical base-equivalent signal. Alternatively, document explicitly in the interface that this flavour intentionally does not emit `Deposit` for non-base deposits so integrators do not assume completeness.

REMEDIATION COMMENT

Addressed in: 5db8b9da2cdabc996fc28617e4dffc134feb5a8f

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by emitting the canonical `IERC4626.Deposit` event with the base-equivalent value in `multiAssetDeposit()` alongside the existing `MultiAssetDeposit` event, so indexers summing `Deposit` no longer under-count non-base deposits.

HAL-005 ACKNOWLEDGED

Silent governance tier reset due to unconditional reinitializer reseeding

Informational · 0.4 AO:S/AC:L/AX:L/R:F/S:U/C:N/A:N/I:H/D:N/Y:N

DESCRIPTION

The `Timelock.reinitializeV3()` function re-runs the full set of `Config.seed…Artifacts()` helpers in `TimelockConfigurationLib`, the same list executed once at genesis inside `initialize()`. These helpers exist to register each protocol selector at its default review tier so that governance actions are subject to the correct timelock delay.

However, every `seed…Artifacts()` helper performs an unconditional `artifactDelayTier[...] = TIER_N` assignment with no read before write guard and emits no event, unlike the sanctioned `Timelock.changeTier()` path, which reverts for an unregistered artifact and emits `ArtifactTierChanged`. When governance has deliberately re-tiered an artifact through `changeTier()` (for example moving a high risk selector to a longer review window after a security review), the next execution of `reinitializeV3()` by the `UPGRADER_ROLE` blindly overwrites that value back to the hardcoded default.

The account holding `UPGRADER_ROLE`, or any upgrade flow that runs the reinitializer, therefore silently discards a prior governance tier decision. The reset produces no `ArtifactTierChanged` event and no per-artifact signal identifying which tiers were reset, so indexers and governance monitors cannot detect that a deliberate review window decision was rolled back to a code default, defeating the auditability the tiered timelock is meant to provide.

The condition is recoverable rather than permanent: because the artifact remains registered, governance can re-apply the intended tier afterward through `Timelock.changeTier()` (self-timelocked at TIER_1). The exposure is also bounded to a single occurrence, since `reinitializeV3()` is guarded by `reinitializer(3)` and executes at most once for the V3 upgrade rather than being a repeatable action.

RECOMMENDATION

Make each `seed…Artifacts()` helper in `TimelockConfigurationLib` gap-filling: read the existing tier and skip the write when it is already set, so `reinitializeV3()` only registers genuinely new artifacts.

Alternatively, route reinitializer-time seeding through a dedicated helper that registers only the artifacts introduced in that release and emit `ArtifactTierChanged` whenever a reseed write changes an existing tier.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Blueprint Finance` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> The intention is not to change any existing tier, but to keep the on-chain registry consistent with the code. Since the timelock was deployed we have added support for several new contract types, so most of these configurations are new registrations rather than updates. For configurations that already exist, no tier is changed, we simply do not want to miss any.

HAL-006 SOLVED

Owner gated controls temporarily unreachable due to unhardened ownership transfer

Informational · 0.4 AO:S/AC:L/AX:L/R:P/S:C/C:N/A:L/I:N/D:L/Y:N

DESCRIPTION

The `OracleModule.transferOwnership()` function overrides OpenZeppelin `Ownable` and gates ownership changes with an `onlyOwner afterDelay` timelock, and `MultiAssetDepositCapHook` inherits `OwnableUpgradeable` through `BaseOwnableHook` to gate its cap and wiring setters. Both concentrate their full administrative surface under a single owner.

However, neither adds a `pendingOwner` / `acceptOwnership` confirmation, so ownership moves in one step to whatever address is supplied, and `MultiAssetDepositCapHook.renounceOwnership()` remains callable. A transfer to a mistyped or unreachable address, or a renounce on the hook, leaves the owner gated functions such as `OracleModule.removeOracle()`, `setDepositCap`, `removeCap`, and `setHookContainer` uncallable.

The condition is recoverable rather than permanent. A stranded cap hook can be replaced or detached through `ConcreteStandardVaultImpl.setHooks` or the role gated `HookContainer.removeHook` / `replaceHook`, and a stranded oracle can be repointed through `configureMultiAsset`, which calls `StateSetterLib.setMultiAssetPricer`.

For `OracleModule`, the `afterDelay` timelock also limits exposure once activated, so the window for an instant mistaken transfer is confined to the pre-activation bootstrap period.

RECOMMENDATION

In `OracleModule.transferOwnership()`, layer OpenZeppelin `Ownable2Step` (or an equivalent `pendingOwner` / `acceptOwnership` step) beneath the existing `afterDelay` gate so a transfer finalizes only after the new owner accepts.

Additionally, in `MultiAssetDepositCapHook`, adopt the same two step transfer and override `renounceOwnership()` to disable it, matching the pattern already used in the codebase.

REMEDIATION COMMENT

Addressed in: 61b64e9fd421e05ee2d68a1949386af6720747b2

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by layering OpenZeppelin `Ownable2Step` beneath the `OracleModule` timelock gate, so ownership finalizes only after the nominated `pendingOwner` accepts, stating:

> OracleModule now layers OpenZeppelin Ownable2Step beneath its timelock gate, so ownership moves only after a proposal matures, nominates a pendingOwner, and that address accepts. The hooks were left on one-step Ownable: a stranded hook is replaced with one setHooks/replaceHook call, whereas a stranded oracle module needs a new module and pricer (the pricer holds it immutable) plus a timelocked reconfiguration and re-seeding of every feed.

HAL-007 ACKNOWLEDGED

Non-base deposit path blocked due to unenforced hook container alignment

• Informational · 0.4 AO:S/AC:L/AX:L/R:F/S:U/C:N/A:H/I:N/D:N/Y:N

DESCRIPTION

The `MultiAssetDepositCapHook.preMultiDeposit()` function records each raw deposit amount against a per token lifetime cap and is guarded by `onlyVaultOrHookContainer`, which accepts forwarded calls only from a single `hookContainer` address the owner sets via `setHookContainer()`. When the vault routes deposits through a `HookContainer`, a non-base deposit succeeds only when this stored address equals the container installed as the vault's hook target.

However, `setHookContainer()` and `StateSetterLib.setHooks()` each validate only that their target reports the same vault, so nothing enforces that the stored `hookContainer` matches the vault's installed `hooks.target`. These two values are written by independent administrative roles, namely the vault `HOOK_MANAGER` and the cap hook owner.

When the two settings diverge, every non-base deposit forwarded by the installed container reverts with `OnlyVaultOrHookContainer`, blocking the entire non-base deposit path while base asset deposits and all other vault functions remain available. The forwarding entrypoint `HookContainer.preMultiDeposit()` is itself `onlyVault`, so only the vault can drive a container and only through the currently installed `hooks.target`; a stale or unrelated container clone can never be invoked to advance the cap hook state on its own.

The condition is recoverable on-chain in a single transaction, since `setHookContainer()` is owner callable at any time and the `HOOK_MANAGER` can realign the target through `setHooks()`.

RECOMMENDATION

In `MultiAssetDepositCapHook`, make `onlyVaultOrHookContainer` accept a forwarded call only when `msg.sender` equals the vault's live hook target read from `IConcreteStandardVaultImpl(VAULT()).getHooks().target`, so the cap hook always tracks the installed pipeline.

Alternatively, keep the stored value but require in `setHookContainer()` that it equals the vault's current hook target.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Blueprint Finance` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> We disagree with the recommended change. The stored `hookContainer` is an explicit trust grant from the cap-hook owner, independent of the vault's `hooks.target`. `preMultiDeposit` advances a monotonic lifetime counter, so the forwarder must be opted into by the hook owner; it must not be implied by `HOOK_MANAGER`. Making the hook track `getHooks().target`, or requiring `setHookContainer` to equal it, lets the vault's hook manager authorize writers of this hook's storage.
> The two contracts are also independently configured by design. `setHooks` accepts any `IHook`, so a vault with a single hook can install that hook directly with no container at all; tying the stored value to the live target assumes a container is always present.
> A mismatch fail-closes only the non-base deposit path and is recoverable through `setHookContainer` or `setHooks`. It is the same class of admin consistency already required to add the hook to the container and install the container on the vault. `setHooks` is additionally timelocked at TIER_2, so a container change is visible 24 hours before it takes effect and either role can correct the mismatch inside that window.

HAL-008 SOLVED

Non-base deposits blocked due to unsynchronized multi asset strategy pointer

Informational · 0.4 AO:S/AC:L/AX:L/R:F/S:U/C:N/A:H/I:N/D:N/Y:N

DESCRIPTION

The `MultiAssetVaultHelperLib.prepareNonBaseDeposit()` function routes every non-base deposit to the single strategy stored in `MASLib.strategy`, re-reading that pointer on each call and requiring the referenced strategy to be `Active` or reverting with `MultiAssetStrategyNotActive`. The pointer is written only by `StateSetterLib.setMultiAssetStrategy()`, which validates the strategy at write time under the `VAULT_MANAGER` role.

However, the registry lifecycle functions `StateSetterLib.removeStrategy()`, `reconcileHaltedStrategy()`, and `toggleStrategyStatus()`, invoked by the `STRATEGY_MANAGER` role, operate only on the `strategyData` registry and never clear or repoint `MASLib.strategy`. Nothing couples the designated pointer to the lifecycle of the strategy it references.

Once the designated strategy is halted, removed, or reconciled, every non-base deposit and `previewDeposit` reverts with `MultiAssetStrategyNotActive`, while base-asset deposits never read `MASLib.strategy` and continue to function.

The `VAULT_MANAGER` can restore the non-base flow at any time by repointing `MASLib.strategy` to another `Active` strategy through `configureMultiAsset`, subject to the 24 hour TIER_2 timelock, so the condition is fully recoverable on-chain; the lifecycle events carry no reference to the pointer, leaving no on-chain signal tying the outage to its cause.

RECOMMENDATION

In `StateSetterLib.removeStrategy()`, `reconcileHaltedStrategy()`, and `toggleStrategyStatus()`, detect when the affected strategy equals `MASLib.strategy`, reset the pointer to `address(0)`, and emit `MultiAssetStrategySet(address(0))` so subsequent non-base deposits fail with the explicit `MultiAssetNotConfigured` error and operators receive a clear on-chain signal.

REMEDIATION COMMENT

Addressed in: 1454aab1cb6bc9043d1190Ode6fd7ecb4e41155f

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by adding a `StateSetterMASLib` overlay that clears `MASLib.strategy` when its strategy is removed and repoints it to the successor when a halted strategy is reconciled, deliberately leaving the reversible halt path untouched.

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.